

Machine Learning – HW#5

General notes:

- It is **recommended** that you work alone to learn the material. However, groups of size 2 are allowed.
- Read and understand the questions, and answer accordingly. Be sure to answer what is asked. When in doubt, you can provide more details.
- Some of the questions require material not covered in class, or will be covered in future lectures. This is intentional.
- **Warning:** do not copy other student work. This includes code similarity. Do not sit and solve assignments together, unless it is your group partner.

Submission document format guidelines:

- Submit pdf document to Gradescope.
- You can use word, latex, or any other editor.
- No need for cover page. Provide your names and IDs in the header.
- Answer each Question on a new page. No need to copy the question, just put Question # at the beginning, then a), b), etc. for each sub-question.
- When submitting to Gradescope, you will be given the chance to map pages to questions.
- For groups, one student submits and adds their teammate during submission on Gradescope.

Question 1 [6 + 14=20 pts]

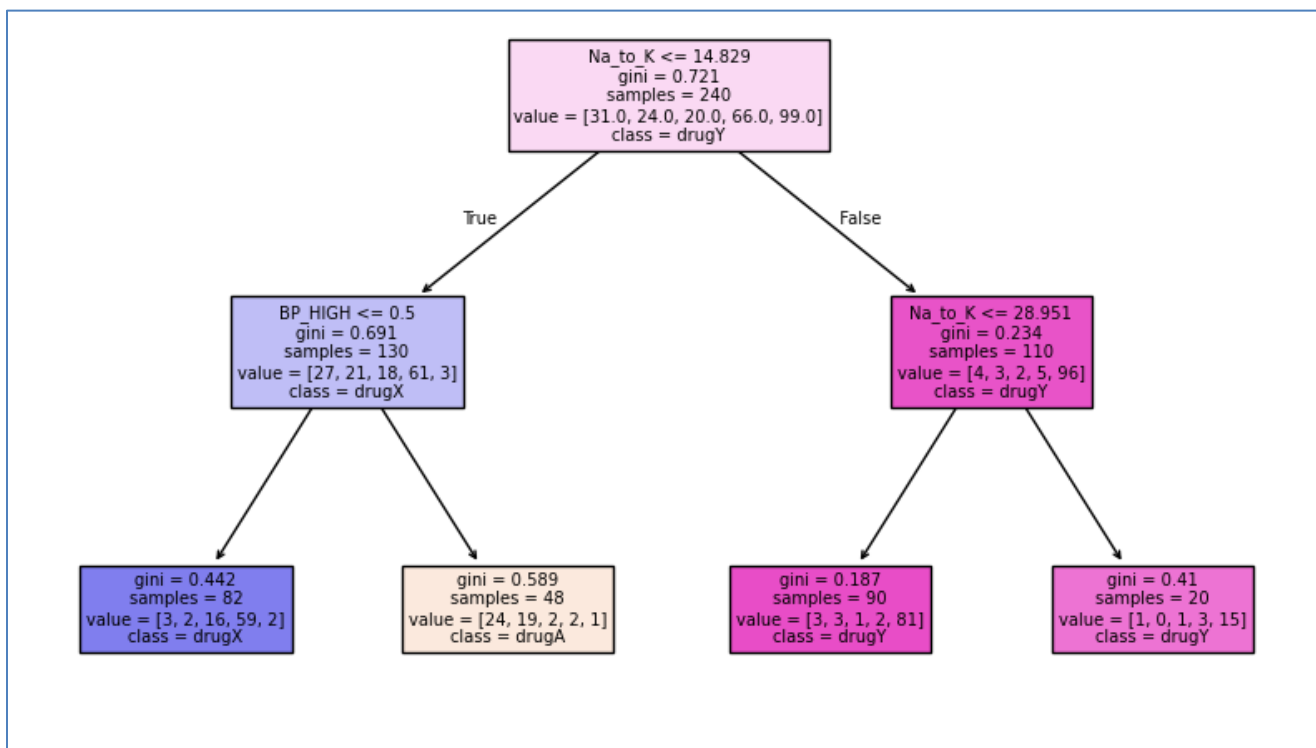
In this question we will play around with decision trees in scikit-learn. You are provided with 'hw5q2_drugs.csv' of a toy dataset adapted from Kaggle ([link](#)). The dataset has been pre-processed for you whereby discrete features have been one-hot encoded.

The dataset contains **label Drug** in the last column, all else are features. The label here is multiclass {0, 1, 2, 3, 4} corresponding to {'drugA', 'drugB', 'drugC', 'drugX', 'drugY'}

NOTE1: for each of the following parts show the *relevant* code *snippet*, plots, and results.

a) Read in the dataset and construct the X feature matrix and y label vector. Then fit a [DecisionTreeClassifier](#) with max_depth=2. After fitting, you can plot the learnt decision tree using [plot_tree](#) function. Reconstruct the figure I have below.

Note that plot_tree uses matplotlib, so you can set the figure size before calling it (set figure size to (10, 8)). Be sure to pass feature names, class names (see ordering above), and set filled to True.



b) As mentioned in lecture, the three hyperparameters (`max_depth`, `min_samples_split`, `min_samples_leaf`) influence the resulting size of the tree. In general, smaller trees are simpler, whereas deeper trees are more complex (tend to overfit). So, let us play around with `max_depth` and see its effect on performance.

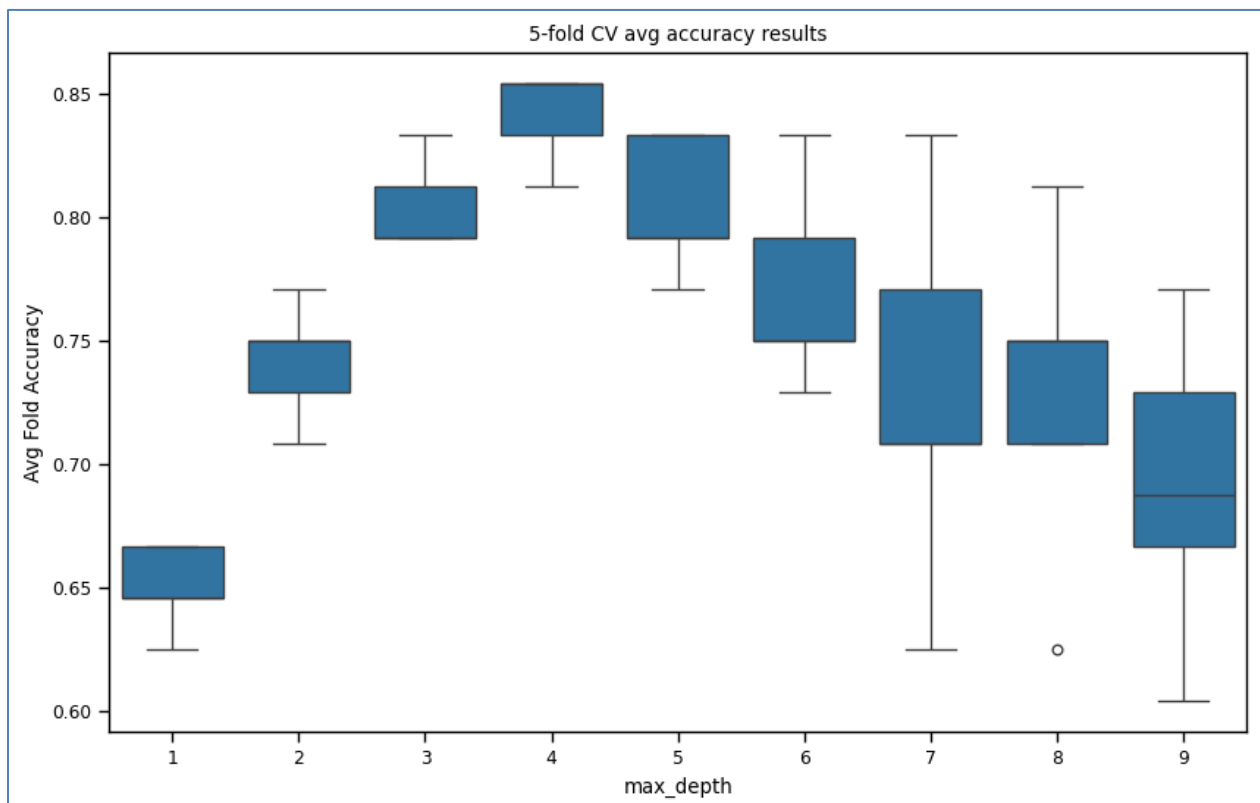
In particular, we want to try the following settings for `max_depth`.
`max_depth = [1, 2, 3, 4, 5, 6, 7, 8, 9]`

Do the following

- Split (`X`, `y`) into 5-folds using stratified sampling. You can use [StratifiedKFold](#) with `shuffle=True` and `random_state=11`.
- In each K-fold iteration, train a decision tree with corresponding `max_depth` from the values above using training folds. Then evaluate on the test fold using [accuracy score](#).
- Plot accuracy boxplots of K-fold performance. Use seaborn's [boxplot](#). Be sure to have the x-axis denote the `max_depth` of the decision tree.

Hint: the model selection & evaluation slides has example code that is helpful.

The following are boxplots I got. As expected, the deeper the tree, the more complex it is and it *may* overfit the training data (in this case it looks like it is).



Question 2 [20 pts]

You are given the stroke **classification** dataset. You worked with this dataset before in previous homework. Here the dataset has been pre-processed for you, and you are given 'hw5q2_stroke.csv'. Also, note that the dataset is much smaller than in previous assignments.

The **label stroke** is the last column (binary {0, 1}), all else are features.

NOTE1: for each of the following parts show the *relevant* code *snippet*, plots, and results.

a) Here we'll do a small experiment to analyze Random Forest (bagging-based method) and AdaBoost performance as we increase the number of learners. Do the following:

- Read in the dataset and construct (X, y).
- Define a list `n_estimators_list = [1, 10, 100, 1000, 5000, 10000, 20000]` which contains the number of learners we will try.
- Do 5-fold CV on (X, y) using [StratifiedKFold](#) with `shuffle=True` and `random_state=11`. In the CV loop, do:
 - Train [RandomForestClassifier](#) and [AdaBoostClassifier](#) with the 7 settings of `n_estimators` in the list above. Be sure to use `random_state=11`. You can do this via an inner loop.
 - For the 7 trained RF and 7 trained AdaBoost, collect the [ROC-AUC](#) and [PR-AUC](#) performance.
- After collecting the results, plot 4 boxplot figures: RF Avg ROC-AUC, RF Avg PR-AUC, AdaBoost ROC-AUC, and AdaBoost PR-AUC.

Hint: the model selection & evaluation slides has example code for the K-fold CV loop is helpful.

Below are the figures I got. Notice that RF reaches a limit with more estimators (no increase in performance), whereas AdaBoost starts to degrade in performance.

